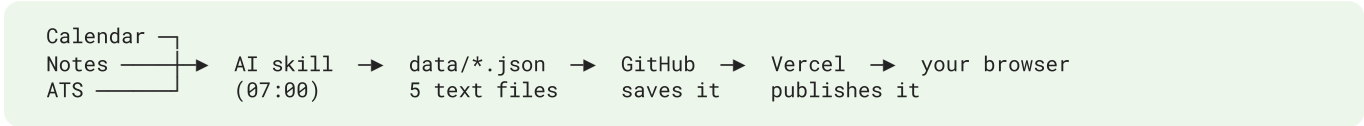


The dashboard with no database

How to get an AI to build, and then keep running, a management dashboard that updates itself every morning. · David Dogan-Webb, Scede

This isn't a list of prompts. Prompts go out of date and work once. This page explains **how the system works and the decisions you have to make**, so you can brief any AI to build it and fix it yourself when it goes wrong. It's deliberately simple, not the best architecture: you can understand all of it in two hours.

The idea, in one picture



PART	WHAT IT DOES	WHY IT'S THERE
The data files	Five small text files hold everything the page shows	No database to set up or pay for. You can open them and read them. Think of a whiteboard, not a filing system
The contract	SCHEMA.md: every field, its allowed values, the rules	The page and the AI agree on the shape. Change the contract, not the code
The skill	A written brief the AI follows each morning: read sources, rewrite the files	The only part that "thinks". Everything else is plumbing
GitHub	Stores the files and every past version	A history of every morning. Yesterday's numbers are one click away
Vercel	Republishes the page whenever GitHub changes	Nobody presses "deploy". Save the files, and the site updates within about a minute
The schedule	Runs the skill at 07:00 on weekdays	A skill you have to remember to run is a skill you'll stop running

The four decisions to make before you type anything

Building the real version of this with live recruitment data, none of the hard parts were code. They were these:

1 · Whose view is it, and at what scale?

One person's pipeline and a 23-person team's are different pages.

Our real team had 98 open roles. The page shows the 15 most urgent, with the real total in the header.

2 · What do your words mean?

"Open role", "hire", "this week": your systems disagree about all of them. Define each one once.

Is a verbal accept a hire? We said no: signed, with a date. Left alone, the AI counted it.

3 · What does "at risk" mean?

If judgement isn't written down as a rule, the AI makes a new one up every morning.

Our first rule flagged roles opened two days ago. We looked at what it flagged and added a grace period.

4 · What must never appear?

Decide before real data goes in, not after.

Counts only, never candidate names. Clients shown as letters, people as numbers.

You need

- **GitHub** and **Vercel** accounts (free; sign in to Vercel with GitHub).
- **An AI that can write code:** Claude or ChatGPT. If it connects to GitHub it saves the files for you; if not, it hands them to you to paste.
- **Copilot?** Go to Shay's session today and try this one later with Copilot Studio.

The order to build it in

1. The page, with example data, plus its SCHEMA.md
2. GitHub, then Vercel: now it's live
3. The skill, run once by hand, and read what it did
4. The schedule, only once a run by hand is right

BRIEFING THE AI

A good brief has four parts

The goal

Whose page, what it's for, what period.

The shape

Point it at the contract (SCHEMA.md) every time.

The rules

Your definitions, your judgement as rules, your non-negotiables.

Done looks like

How you'll know in ten seconds that it worked.

One real example, taken apart. These are excerpts from the brief that runs our live version. On the left is what it says; on the right, why that line is there.

Team shown: everyone who reports to [manager], still employed. Quarter: the calendar quarter containing today.	Goal. Without an exact definition, every run quietly picks a different team or period.
Open role = live in the ATS, not closed or cancelled. Hire = signed, with a signed date. A verbal accept is not a hire.	Definitions. Pin down every word your systems disagree on. Once, in writing.
No due date in the meeting notes → leave it blank. Do not guess one. No target set → blank plus a note, never 0.	Missing data. Fake data is always complete; real data never is. Say what to do with gaps, or the AI fills them in.
At risk = nothing logged for 4 weeks and open 60+ days. Watch = open 90+ days, or 21+ days with no interviews.	Judgement as a rule. The same every morning, and something your team can argue with.
Counts only. No candidate names, ever. Run the check before saving. If it fails, fix the data, never the check.	Non-negotiables, enforced. A rule in a prompt is a request; a check is a guarantee.
Reply in five lines: sources used or missing · meetings · actions · roles needing attention · published or not.	Done looks like. You can tell at a glance whether the run worked.

What went wrong, or nearly did

- **Real data has gaps; fake data doesn't.** A blank due date would have crashed the to-do list, and a missing target would have printed "NaN". We only caught it because we tested with real data first. So should you.
- **Gaps get filled in.** Meeting notes rarely give due dates. Unless you say "leave it blank", the AI will invent one.
- **Private data slipped in.** An email address got into the sources line, and the check caught it. That's what the check is for.
- **The first rule was wrong.** "No interviews in 4 weeks = watch" flagged roles opened that week. Read what a rule flags before you trust it.
- **The first run raised a real question.** 19 of 98 roles had nothing logged for a month. Is the search stalled, or did nobody close the role?

If your laptop is locked down

- **AI can't reach GitHub:** ask for the files and paste them into GitHub's web editor.
- **AI can't reach your ATS:** export a report each morning and attach it.
- **AI can't see your calendar:** paste today's agenda.
- **No GitHub account allowed:** Shay's session needs nothing external.

Take it further, in this order

1. Add a section: candidates waiting on hiring-manager feedback.
2. Swap the export for a live connection to the source.
3. Put it behind a login **before** real data goes in.
4. Then a database, once more than one person writes.

Example repository

github.com/daviddw-scede/bbb-management-dashboard

Live example

bbb-management-dashboard-daviddw-6234s-projects.vercel.app

Show and tell

14:00. Bring your URL, not your slides.

QR to the
live example
goes here